PROJECT STATUS → EXECUTION PLAN

Veridex

Phase 2+ Development Plan

Turning a well-scaffolded agent orchestration platform into a running system

WHAT THIS DOCUMENT COVERS

- **5 build phases**, ordered by dependency — orchestration first, then knowledge wiring, governance, delivery, hardening
- **Repository-grounded scope,** based on actual line counts and file structure, not just the stated roadmap
- **IDE-ready instructions** for each phase: what to run, what "done" looks like, and how to avoid rework
- **A resource budget section** with lower-cost alternatives at each phase, to avoid burning credits or cloud spend

1. Repository Reality Check

Before planning new work, it's worth being precise about what's already real. The status report describes the project in terms of intent (“engineered,” “integrated,” “configured”) which can blur the line between a working feature and a well-named stub. Extracting the archive and measuring each backend module directly gives a sharper picture:

Module	Lines	Status	What this means
knowledge/	2,057	BUILT OUT	The most substantial module by far. Ingestion + vector/graph logic has real depth.
api/	605	SCAFFOLDED	Route signatures exist; depth of actual handler logic needs verification per-route.
ai/	560	SCAFFOLDED	Router, providers, memory, prompts present as structure — execution wiring is thin.
agents/	304	SKELETON	Orchestrator imports Planner/Scheduler/Bus correctly, but methods are short stubs.
connectors/	227	SKELETON	BaseConnector pattern is sound; per-source connectors are likely partial implementations.
core/	220	SKELETON	Config and shared utilities — normal to be lean at this stage.
observability/	198	SKELETON	Telemetry hooks exist; trace recording end-to-end is not yet proven.
db/	175	SKELETON	Models are defined (confirmed by Alembic migrations existing).
governance/	73	STUB	audit.py, policies/, retention.py exist as files but with minimal logic.
workers/	59	STUB	celery_app.py is correctly configured with two real queues (planner, agent).
plugins/	29	STUB	Minimal.
schemas/	23	STUB	Very few Pydantic models defined so far.
security/	33	STUB	Minimal beyond what OAuth wiring needs.
events/	0	EMPTY	Directory exists; no implementation yet.
repositories/	0	EMPTY	Directory exists; no implementation yet.

Module	Lines	Status	What this means
services/	0	EMPTY	Directory exists; no implementation yet.
utils/	0	EMPTY	Directory exists; no implementation yet.

Also confirmed directly: **8 test files** exist against roughly 4,500+ lines of backend scaffold, a CI workflow (`.github/workflows/ci.yml`) is already in place, and `backend/pyproject.toml` lists real, current dependency versions (FastAPI 0.139, SQLAlchemy 2.0.51, Celery via Redis, Qdrant client 1.18, Neo4j driver 6.2) — so the environment can genuinely be installed and run today, even before new code is written.

WHY THIS MATTERS FOR PLANNING

This changes the plan in one important way: Phase 1 is not “build an orchestrator from scratch.” It's **closing the gap between a correctly-imported class structure and a class that actually runs**. That's a smaller, more precise task — and it's why Phase 1 below is scoped to specific method bodies, not new architecture.

2. How to Read This Plan

This plan is organized as five phases in strict dependency order. Each phase only depends on the phase(s) before it — nothing here asks you to build two things in parallel that don't yet talk to each other. Every phase has the same five-part structure so you can jump straight to the part you need in your IDE:

Objective	the single sentence that defines “done” for the phase
Why this order	why it can't move earlier or later in the sequence
Build checklist	concrete files/classes to touch, grounded in the actual repo structure
Definition of done	a testable condition, not a vibe — something you can verify locally
Credit / cost notes	the cheapest way to validate the phase before scaling anything up

One deliberate choice: **this plan does not add new infrastructure dependencies.** Everything proposed uses what's already declared in `backend/pyproject.toml` and the existing Docker/Helm setup. New scope creep (a new message queue, a second vector store, a rewrite in another framework) is explicitly out of scope until Phase 5, and only then as an optional hardening step — not a requirement.

or Run

cute real Planner → Worker agent cycles, end to end, on one goal.

Objective

Take `Orchestrator.execute_goal()` from a correctly-imported skeleton to a function that can accept a plain-text goal, decompose it via `PlanningEngine`, push tasks onto the Celery queues already defined in `celery_app.py`, and return a completed result — for one simple goal, run locally, end to end.

Why this order

Every other objective in the original report (governance enforcement, evaluation, frontend consumption) assumes there's something running to govern, evaluate, or display. The orchestrator is the load-bearing wall. Building anything else first means building against a system that can't yet execute a task, which invites rework the moment Phase 1 lands.

Build checklist

- **PlanningEngine.decompose()** (`app/agents/planner/engine.py`) — implement the actual task-graph construction from a goal string into `TaskNode` objects. Start with a fixed decomposition strategy (e.g. single-step or simple sequential split) before adding LLM-driven planning — this de-risks the queue plumbing separately from planning quality.
- **TaskScheduler** (`app/agents/scheduler/queue.py`) — connect this to the existing `planner_queue` / `agent_queue` Celery routes rather than building new routing logic.
- **app/workers/tasks/planner.py** and **agent.py** — these files already exist; give them real task bodies that call into the agent execution module and return results Celery can serialize as JSON.
- **AgentBus** (`app/agents/communication/bus.py`) — implement the minimum pub/sub needed for the Orchestrator to know when a Worker task completes, using Redis (already a dependency) rather than introducing a new broker.
- **execution/** module — wire one real Worker agent that can call an LLM via the existing `litellm` router in `ai/router.py`, so the full loop touches a real model call, not a mock.
- Add 2–3 tests under `backend/tests/` (there are 8 there already — follow existing patterns) that assert a full goal-to-result cycle completes without manual intervention.

Definition of done

VERIFIABLE OUTCOME

From a local shell: `celery -A app.workers.celery_app worker` is running, a goal is submitted (via a script or a temporary API route), and within seconds a completed task result appears — observable either in Redis directly or via a log line — with no unhandled exceptions. This should be reproducible on a fresh clone with nothing more than Docker Compose up plus one command.

Credit / cost notes

This entire phase can be validated with a single cheap or free-tier LLM call per test run (or a mocked LLM response while the plumbing is being debugged). There is no reason to run this against a large model or in a cloud environment yet — Docker Compose locally is sufficient, and it's the fastest feedback loop besides.

o the Pipeline

/Gmail ingestion → chunking → embeddings → Qdrant + Neo4j.

Objective

Take one connector — Filesystem is the lowest-risk starting point since it needs no OAuth — all the way from raw document ingestion through chunking and embedding into a queryable Qdrant collection, with corresponding nodes written into Neo4j. Prove the full path once before repeating it for the OAuth-backed connectors (GitHub, Gmail, Jira, Notion, Slack).

Why this order

The `knowledge/` module is already the most substantial part of the codebase (2,057 lines) — it's the natural second phase because it's the closest to finished and because agents built in Phase 1 are far more useful once they can retrieve real context instead of operating blind.

Build checklist

- **Filesystem connector** — confirm `full_sync` and `incremental_sync` against `BaseConnector` actually yield document blobs + metadata as designed; this is the fastest connector to validate without network dependencies or OAuth setup.
- **Text splitter** — add a chunking step (token-aware, using the existing `tiktoken` dependency) between raw ingestion and embedding.
- **Embedding call** — route through the existing `litellm` abstraction in `ai/providers` rather than calling an embedding API directly, to keep provider-swapping possible later.
- **Qdrant write path** — confirm collection creation, upsert, and a basic similarity query work against a local Qdrant Docker container.
- **Neo4j write path** — create a minimal graph schema (Document → Chunk → Entity, or whatever relations the `knowledge/` module already implies) and confirm nodes persist correctly.
- Repeat the proven path for one OAuth-backed connector (GitHub is a reasonable second choice — OAuth pipeline is already integrated per the status report) to confirm the pattern generalizes.

Definition of done

VERIFIABLE OUTCOME

A document dropped into a watched local folder is ingested, chunked, embedded, and becomes retrievable via a similarity search against Qdrant — and a corresponding node exists in Neo4j — without any manual step beyond the initial trigger. This should be scriptable as an end-to-end smoke test.

Credit / cost notes

Embedding calls are typically the cheapest category of LLM usage — use a small/local embedding model if one is available through `litellm`, and test against a handful of small local files (a few KB each) rather than a real Slack/Jira export while validating plumbing. Save OAuth-backed connector testing for after the Filesystem path

is fully proven, since debugging OAuth issues and ingestion-logic issues at the same time is slower, not just costlier.

y Enforcement

to live FastAPI middleware enforcing the existing RBAC model.

Objective

Implement OPA-style policy evaluation as FastAPI middleware that reads YAML policy files and enforces them against the `Organization / Workspace / Project / User / Role / Permission` models that already exist in the database layer.

Why this order

Governance only has something to govern once agents can act (Phase 1) and retrieve data (Phase 2). Building policy enforcement earlier means testing it against no real traffic — there's nothing to verify a policy "blocks" if there's no action for it to block yet.

Build checklist

- **governance/policies/** — define a small starter set of YAML policies (e.g. "Workspace X members cannot invoke connector Y") that map directly onto the existing RBAC tables, rather than inventing a parallel permission model.
- **Policy evaluation middleware** — add a FastAPI dependency/middleware that loads and evaluates these YAML rules per-request, short-circuiting with a 403 on violation.
- **governance/audit.py** — extend this into an append-only audit log entry per policy decision (allow/deny), which directly satisfies the "immutable audit ledgers" objective from the original report.
- **governance/retention.py** — wire a basic retention rule (e.g. audit logs retained N days) so the module has one real, testable behavior rather than being purely structural.
- Add tests confirming: an allowed action passes through, a denied action is blocked with a clear 403, and both produce an audit log entry.

Definition of done

VERIFIABLE OUTCOME

Two test users with different roles hit the same endpoint; one succeeds, one receives a 403 that traces directly to a specific YAML policy line, and both attempts are visible in the audit log — with no manual database inspection required to see why the second one failed.

Credit / cost notes

This phase is essentially free to build and test — it's request routing and database logic, with no LLM calls involved. It's a good phase to spend more IDE time on precisely because it doesn't consume any paid inference at all.

ration

xisting Next.js frontend and the scaffolded CLI — no new UI

Objective

Connect the frontend (`frontend/src/app`) and CLI (`cli/`) to the now-functional backend: submit a goal, watch it execute, see retrieved knowledge, and see a governance decision — all three previous phases visible in one place.

Why this order

There's now something worth displaying. Building UI before this point means building against endpoints that return mock or empty data, which typically has to be redone once real responses have a different shape than assumed.

Build checklist

- **API client layer** (`frontend/src/lib`) — a typed client wrapping the real `backend/app/api` routes; generate types from the FastAPI OpenAPI schema if possible rather than hand-writing them, since FastAPI produces this automatically.
- **Goal submission view** — a single page/component that submits a goal and polls or streams (FastAPI's `sse-starlette` dependency is already present) for the result.
- **CLI parity** (`cli/`) — mirror the same goal-submission flow as a command, since it's the faster path for verifying backend behavior during development regardless of frontend state.
- **SDK usage** — have the frontend/CLI go through the `sdk/python/veridex` package where practical, which both exercises the SDK and keeps client logic in one place.

Definition of done

VERIFIABLE OUTCOME

A goal typed into the frontend (or CLI) results in a visible, real execution trace — not a mock response — including at least one retrieved knowledge snippet and one governance check, viewable without opening a database client.

Credit / cost notes

Frontend/CLI work by itself costs nothing beyond normal development time. The only cost driver is how often you re-trigger full goal executions while testing the UI — mock the API response shape locally once it's known, and only hit the real backend for final verification passes.

ation & Hardening

ge evaluation, load testing, and chaos scripts — in that order.

Objective

With a working system in place, build the observability and evaluation layer the original report describes as a Phase 1 baseline objective, plus exercise the existing `backend/tests/load` and `scripts/chaos` directories that are currently unused scaffolding.

Why this order

Tracing and evaluation are meaningless without real executions to trace and evaluate. This is deliberately last: it's the phase most tempting to front-load (observability feels foundational) but it produces the least value until Phases 1–4 give it something real to observe.

Build checklist

- **ai/telemetry** — wire trace recording so each Phase-1 goal execution produces a full DAG trace (Planner decomposition → each Worker task → final result), stored in a form the frontend (Phase 4) can eventually visualize.
- **ai/evaluation** — implement one LLM-as-Judge evaluator that scores a completed goal execution against a simple rubric; start with a single evaluation dimension (e.g. “did the result answer the goal”) before adding more.
- **backend/tests/load** — run the existing load test scaffolding against the now-working orchestrator to find real bottlenecks, rather than load-testing stub endpoints.
- **scripts/chaos** — exercise the existing chaos scripts against the real Celery/Redis setup locally before considering the Istio retry configuration in `kubernetes/istio`, which is a cloud-deployment concern and can wait.
- **Versioned prompts** — once evaluation exists, add basic prompt versioning so evaluation results can be compared across prompt changes over time.

Definition of done

VERIFIABLE OUTCOME

A completed goal execution produces a full trace and at least one evaluation score, both retrievable after the fact — and a local load test run against the real orchestrator produces a concrete throughput/latency number rather than an untested assumption.

Credit / cost notes

This is the most credit-sensitive phase, since LLM-as-Judge evaluation means a second model call for every execution being evaluated. Evaluate on a small fixed batch of goals (10–20) rather than every execution during development, and keep load testing scoped to a handful of concurrent goals locally — there's no need to simulate production-scale traffic before the system has run in production at all.

Cross-Cutting: Empty Modules to Resolve

Four directories exist in the repository with zero lines of implementation: `events/`, `repositories/`, `services/`, and `utils/`. These aren't a phase on their own — they should be filled in as each phase above needs them, rather than pre-built speculatively. Here's where each one naturally gets its first real content:

Module	Fill during	First real use
events/	Phase 1	Task-completion events published on the AgentBus need an event schema/dispatcher — this is the natural home for it.
repositories/	Phase 2	Data-access layer between the ingestion pipeline and Postgres/Qdrant/Neo4j; keeps query logic out of connector and API code.
services/	Phase 3	Policy-evaluation and audit-logging business logic belongs in a service layer, not directly in middleware.
utils/	Ongoing	Shared helpers (chunking utilities, retry wrappers, serialization) — add opportunistically as duplication appears; resist pre-populating this speculatively.

The reasoning here matters more than the assignment: an empty directory with a good name is easy to mistake for “architecture already decided.” Filling each one only when a concrete need appears in the phase above avoids building an abstraction layer for a use case that turns out, once real code exists, to want a different shape.

Running This Without Burning Credits

The single biggest cost lever across all five phases is the same one: **how much of the work genuinely needs a live LLM call to validate.** Most of Phases 1–4 don't — they're plumbing, database, and API work that can be fully verified with mocked or free-tier model responses. Only specific, narrow slices actually need real inference:

Genuinely needs live inference

- Confirming `ai/router.py` correctly calls a real provider through litellm (Phase 1) — needed once, not per test run
- Embedding a handful of test documents to confirm the Qdrant write path (Phase 2) — embeddings are typically the cheapest inference category available
- LLM-as-Judge scoring in Phase 5, which is the only place a second model call per execution is unavoidable

Doesn't need live inference at all

- Celery/Redis queue wiring, task routing, and result serialization (Phase 1 plumbing)
- Chunking, Qdrant/Neo4j write paths, connector sync logic (Phase 2 plumbing)
- All of Phase 3 — governance is pure request/database logic with zero model calls
- Frontend/CLI wiring against a known API shape (Phase 4) — mock the response once it's known
- Load and chaos testing infrastructure itself (Phase 5) — only the workload being tested may call a model

Practical setup for local, low-cost development

- Run Postgres, Redis, Qdrant, Neo4j, and MinIO via the existing Docker Compose setup implied by the stack — all four are free to run locally and none require a paid account to develop against.
- Use `litellm`'s provider abstraction to point at a small, cheap, or local model during development, and only switch to a larger model for final verification passes — the abstraction already in `ai/router.py` makes this a config change, not a code change.
- Mock the LLM call entirely (a fixed canned response) while debugging queue/plumbing issues in Phase 1 — a broken Celery task is a Celery problem, not a model problem, and doesn't need a real API call to diagnose.
- Keep the Helm/Kubernetes/Istio deployment (already scaffolded under `helm/` and `kubernetes/istio`) out of scope until Phase 5 at the earliest — cloud infrastructure costs real money and there's nothing to deploy yet that local Docker Compose can't prove first.

Suggested Order of Operations

A single-page checklist for pasting directly into an IDE task list or project board:

#	Task	Phase
1	Implement PlanningEngine.decompose() with a fixed/simple strategy	1
2	Wire app/workers/tasks/planner.py and agent.py to real logic	1
3	Implement minimal AgentBus pub/sub over Redis	1
4	Prove one full goal → result cycle locally, add tests	1
5	Validate Filesystem connector full/incremental sync	2
6	Add chunking step using tiktoken between ingestion and embedding	2
7	Confirm Qdrant + Neo4j write paths against real (small) data	2
8	Repeat proven path for one OAuth connector (GitHub)	2
9	Write starter YAML policies mapped to existing RBAC tables	3
10	Add policy-evaluation middleware + audit log entries	3
11	Build typed API client from FastAPI's OpenAPI schema	4
12	Ship goal-submission view in frontend + matching CLI command	4
13	Wire DAG trace recording for Phase-1 executions	5
14	Implement one LLM-as-Judge evaluation dimension	5
15	Run existing load/chaos scripts against the real system	5

A NOTE ON SCOPE

Every item above builds on what's genuinely in the repository today — no new frameworks, no new infrastructure dependencies, and no rewrites of the substantial existing work in `knowledge/`. The goal of this ordering is a system that runs end to end as early as possible (Phase 1), with each subsequent phase adding one more real capability on top of something that already works, rather than several partially-built subsystems that only connect at the very end.